

# Python Programming Tutorial

## PYTHON

### Python

Python is a simple, easy to learn, powerful, high level and object-oriented programming language. Python is an interpreted scripting language also. Guido Van Rossum is known as the founder of python programming. Python is a popular programming language. It was created in 1991.

### Introduction to Python

It covers the topics such as python programming, features, history, versions, how to install, example, how to execute, variables, keywords, identifiers, literals, operators and comments.

### Control Statement

The control statement in python covers if statement, for loop, while loop, break statement, continue statement and pass statement.

### Python Strings

The string chapter in python provides the full functionality to work on strings such as accessing string, applying string operators, details of slice notation, applying different functions etc.

### Python Lists

The list chapter in python covers the data structure part such as storing data in list, accessing data, manipulating data etc.

### Python Tuples

A sequence of immutable objects is known as tuple, it covers accessing tuple, adding tuple, replacing tuple, updating tuple etc.

### Python Dictionary

The python dictionary provides details about dictionary operations.

### Python Functions

It provides a list of python functions with its implementations.

### Python Files I/O

How to write data into file and read data from file in python?

### Python Modules

What is python module? What are the usage of modules?

# Python Programming Tutorial

## Python Exceptions

It explains the errors and exceptions in python.

## PYTHON INTRODUCTION

Python is a general purpose, dynamic, high level and interpreted programming language. It supports Object Oriented programming approach to develop applications. It is simple and easy to learn and provides lots of high-level data structures.

Python is easy to learn yet powerful and versatile scripting language which makes it attractive for Application Development.

Python's syntax and dynamic typing with its interpreted nature, makes it an ideal language for scripting and rapid application development.

Python supports multiple programming pattern, including object oriented, imperative and functional or procedural programming styles.

Python is not intended to work on special area such as web programming. That is why it is known as multipurpose because it can be used with web, enterprise, 3D CAD etc.

We don't need to use data types to declare variable because it is dynamically typed so we can write `a=10` to assign an integer value in an integer variable.

Python makes the development and debugging fast because there is no compilation step included in python development and edit-test-debug cycle is very fast.

## PYTHON FEATURES

Python provides lots of features that are listed below.

### **1) Easy to Learn and Use**

Python is easy to learn and use. It is developer-friendly and high level programming language.

### **2) Expressive Language**

Python language is more expressive means that it is more understandable and readable.

### **3) Interpreted Language**

Python is an interpreted language i.e. interpreter executes the code line by line at a time. This makes debugging easy and thus suitable for beginners.

### **4) Cross-platform Language**

# Python Programming Tutorial

Python can run equally on different platforms such as Windows, Linux, Unix and Macintosh etc. So, we can say that Python is a portable language.

## 5) Free and Open Source

Python language is freely available at official web address. The source-code is also available. Therefore it is open source.

## 6) Object-Oriented Language

Python supports object oriented language and concepts of classes and objects come into existence.

## 7) Extensible

It implies that other languages such as C/C++ can be used to compile the code and thus it can be used further in our python code.

## 8) Large Standard Library

Python has a large and broad library and provides rich set of module and functions for rapid application development.

## 9) GUI Programming Support

Graphical user interfaces can be developed using Python.

## 10) Integrated

It can be easily integrated with languages like C, C++, JAVA etc.

## PYTHON HISTORY

Python laid its foundation in the late 1980s.

The implementation of Python was started in the December 1989 by Guido Van Rossum at CWI (Centrum Wiskunde & Informatica) in Netherland.

In February 1991, van Rossum published the code (labeled version 0.9.0) to alt.sources.

In 1994, Python 1.0 was released with new features like: lambda, map, filter, and reduce.

Python 2.0 added new features like: list comprehensions, garbage collection system.

On December 3, 2008, Python 3.0 (also called "Py3K") was released. It was designed to rectify fundamental flaw of the language.

ABC programming language is said to be the predecessor of Python language which was capable of Exception Handling and interfacing with Amoeba Operating System.

# Python Programming Tutorial

Python is influenced by following programming languages:

ABC language.

Modula-3

## **PYTHON APPLICATIONS AREA**

Python is known for its general purpose nature that makes it applicable in almost each domain of software development. Python as a whole can be used in any sphere of development.

Here, we are specifying applications areas where python can be applied.

### **1) Web Applications**

We can use Python to develop web applications. It provides libraries to handle internet protocols such as HTML and XML, JSON, Email processing, request, BeautifulSoup, Feedparser etc. It also provides Frameworks such as Django, Pyramid, Flask etc to design and develop web based applications. Some important developments are: PythonWikiEngines, Pocoo, PythonBlogSoftware etc.

### **2) Desktop GUI Applications**

Python provides Tk GUI library to develop user interface in python based application. Some other useful toolkits wxWidgets, Kivy, pyqt that are useable on several platforms. The Kivy is popular for writing multitouch applications.

### **3) Software Development**

Python is helpful for software development process. It works as a support language and can be used for build control and management, testing etc.

### **4) Scientific and Numeric**

Python is popular and widely used in scientific and numeric computing. Some useful library and package are SciPy, Pandas, IPython etc. SciPy is group of packages of engineering, science and mathematics.

### **5) Business Applications**

Python is used to build Bussiness applications like ERP and e-commerce systems. Tryton is a high level application platform.

### **6) Console Based Application**

We can use Python to develop console based applications. For example: IPython.

### **7) Audio or Video based Applications**

# Python Programming Tutorial

Python is awesome to perform multiple tasks and can be used to develop multimedia applications. Some of real applications are: TimPlayer, cplay etc.

## 8) 3D CAD Applications

To create CAD application Fandango is a real application which provides full features of CAD.

## 9) Enterprise Applications

Python can be used to create applications which can be used within an Enterprise or an Organization. Some real time applications are: OpenErp, Tryton, Picalo etc.

## 10) Applications for Images

Using Python several application can be developed for image. Applications developed are: VPython, Gogh, imgSeek etc.

There are several such applications which can be developed using Python

## PYTHON VARIABLES

Variable is a name which is used to refer memory location. Variable also known as identifier and used to hold value.

In Python, we don't need to specify the type of variable because Python is a type infer language and smart enough to get variable type.

Variable names can be a group of both letters and digits, but they have to begin with a letter or an underscore.

It is recommended to use lowercase letters for variable name. Rahul and rahul both are two different variables.

**Note - Variable name should not be a keyword.**

Declaring Variable and Assigning Values

Python does not bound us to declare variable before using in the application. It allows us to create variable at required time.

We don't need to declare explicitly variable in Python. When we assign any value to the variable that variable is declared automatically.

The equal (=) operator is used to assign value to a variable.

Eg:

a=10

# Python Programming Tutorial

```
name="ravi"  
salary=15000  
print( a)  
print( name)  
print( salary)
```

## Multiple Assignment

Python allows us to assign a value to multiple variables in a single statement which is also known as multiple assignment.

We can apply multiple assignments in two ways either by assigning a single value to multiple variables or assigning multiple values to multiple variables. Let's see given examples.

### 1. Assigning single value to multiple variables

Eg:

```
x=y=z=50  
print( x)  
print( y)  
print( z)
```

**Output:**

```
>>>  
50  
50  
50  
>>>
```

### 2. Assigning multiple values to multiple variables:

Eg:

```
a,b,c=5,10,15  
print( a)  
print( b)  
print( c)
```

**Output:**

```
>>>  
5  
10  
15  
>>>
```

The values will be assigned in the order in which variables appears.

# Python Programming Tutorial

## PYTHON OPERATORS

Operators are particular symbols that are used to perform operations on operands. It returns result that can be used in application.

### Example

4 + 5 = 9

Here 4 and 5 are Operands and (+) , (=) signs are the operators. This expression produces the output 9.

### Types of Operators

Python supports the following operators

- Arithmetic Operators.
- Relational Operators.
- Assignment Operators.
- Logical Operators.
- Membership Operators.
- Identity Operators.

### Arithmetic Operators

The following table contains the arithmetic operators that are used to perform arithmetic operations.

| Operators | Description                                                |
|-----------|------------------------------------------------------------|
| //        | Perform Floor division(gives integer value after division) |
| +         | To perform addition                                        |
| -         | To perform subtraction                                     |
| *         | To perform multiplication                                  |
| /         | To perform division                                        |
| %         | To return remainder after division(Modulus)                |
| **        | Perform exponent(raise to power)                           |

### Relational Operators

The following table contains the relational operators that are used to check relations.

| Operators | Description  |
|-----------|--------------|
| <         | Less than    |
| >         | Greater than |

# Python Programming Tutorial

|    |                             |
|----|-----------------------------|
| <= | Less than or equal to       |
| >= | Greater than or equal to    |
| == | Equal to                    |
| <> | Not equal to(similar to !=) |

## Assignment Operators

The following table contains the assignment operators that are used to assign values to the variables.

| Operators | Description               |
|-----------|---------------------------|
| =         | Assignment                |
| /=        | Divide and Assign         |
| +=        | Add and assign            |
| -=        | Subtract and Assign       |
| *=        | Multiply and assign       |
| %=        | Modulus and assign        |
| **=       | Exponent and assign       |
| //=       | Floor division and assign |

## Logical Operators

The following table contains the arithmetic operators that are used to perform arithmetic operations.

| Operators | Description                                                    |
|-----------|----------------------------------------------------------------|
| and       | Logical AND(When both conditions are true output will be true) |
| or        | Logical OR (If any one condition is true output will be true)  |
| not       | Logical NOT(Compliment the condition i.e., reverse)            |

### Example

```
a=5>4 and 3>2
```

```
print( a)
```

```
b=5>4 or 3<2
```

```
print( b)
```

```
c=not(5>4)
```

```
print( c)
```

### Output:

```
>>>
```

```
True
```

```
True
```

# Python Programming Tutorial

False

>>>

## Membership Operators

The following table contains the membership operators.

| Operators | Description                                                                    |
|-----------|--------------------------------------------------------------------------------|
| in        | Returns true if a variable is in sequence of another variable, else false.     |
| not in    | Returns true if a variable is not in sequence of another variable, else false. |

### Example

```
a=10
b=20
list=[10,20,30,40,50]
if (a in list):
    print( "a is in given list")
else:
    print( "a is not in given list")
if(b not in list):
    print( "b is not given in list")
else:
    print( "b is given in list")
```

### Output:

```
>>>
a is in given list
b is given in list
>>>
```

## Identity Operators

The following table contains the identity operators.

| Operators | Description                                                        |
|-----------|--------------------------------------------------------------------|
| is        | Returns true if identity of two operands are same, else false      |
| is not    | Returns true if identity of two operands are not same, else false. |

### Example

```
a=20
```

# Python Programming Tutorial

```
b=20
if( a is b):
    print( "a, b have same identity" )
else:
    print( "a, b are different" )
b=10
if( a is not b):
    print( " a, b have different identity" )
else:
    print( "a,b have same identity " )
```

## Output

```
>>>
a, b have same identity
a, b have different identity
>>>
```

## PYTHON COMMENTS

Python supports two types of comments:

### 1) Single lined comment:

In case user wants to specify a single line comment, then comment must start with #?

Eg:

```
# This is single line comment.
```

### 2) Multi lined Comment:

Multi lined comment can be given inside triple quotes.

eg:

```
""" This
    Is
    Multiline comment"""
```

eg:

```
#single line comment
print( "Hello Python")
"""This is
multiline comment"""
```

# Python Programming Tutorial

## PYTHON IF STATEMENTS

The Python if statement is a statement which is used to test specified condition. We can use if statement to perform conditional operations in our Python application.

The if statement executes only when specified condition is true. We can pass any valid expression into the if parentheses.

There are various types of if statements in Python.

if statement

if-else statement

nested if statement

### **Python if Syntax**

```
if(condition):  
    statements
```

### **Example**

```
a=10  
if (a==10):  
    print( "Welcome to javatpoint" )
```

### **Output:**

Hello User

### **Python If Else Syntax**

```
if(condition): False  
    statements  
else: True  
    statements
```

### **Example-**

```
year=2000  
if (year%4==0):  
    print( "Year is Leap"  
else:  
    print( "Year is not Leap"
```

### **Python Nested If Else Syntax**

```
If (statement):  
    Body  
elif (statement):  
    Body  
else:  
    Body
```

### **Example**

# Python Programming Tutorial

```
a=10
if (a>=20):
    print( "Condition is True")
else:
    if (a>=15):
        print( "Checking second value" )
    else:
        print( "All Conditions are false")
```

## Output:

All Conditions are false.

## FOR LOOP

Python for loop is used to iterate the elements of a collection in the order that they appear. This collection can be a sequence(list or string).

## Python For Loop Syntax

for <variable> in <sequence>:

## Explanation:

Firstly, the first value will be assigned in the variable.

Secondly all the statements in the body of the loop are executed with the same value.

Thirdly, once step second is completed then variable is assigned the next value in the sequence and step second is repeated.

Finally, it continues till all the values in the sequence are assigned in the variable and processed.

## Example

```
num=2
for a in range (1,6):
    print( num * a )
```

## Output:

```
2
4
6
8
10
```

## Python Example to Find Sum of 10 Numbers

```
sum=0
for n in range(1,11):
    sum+=n
```

# Python Programming Tutorial

```
print( sum )
```

**Output:**

```
55
```

## Python Nested For Loops

Loops defined within another Loop are called Nested Loops. Nested loops are used to iterate matrix elements or to perform complex computation.

When an outer loop contains an inner loop in its body it is called Nested Looping.

## Python Nested For Loop Syntax

```
for <expression>:  
    for <expression>:  
        Body
```

### Example

```
for i in range(1,6):  
    for j in range (1,i+1):  
        print( i,end='')  
    print( )
```

**Output:**

```
>>>
```

```
1
```

```
2 2
```

```
3 3 3
```

```
4 4 4 4
```

```
5 5 5 5 5
```

```
>>>
```

### Explanation:

For each value of Outer loop the whole inner loop is executed.

For each value of inner loop the Body is executed each time.

## Python Nested Loop Example 2

```
for i in range (1,6):  
    for j in range (5,i-1,-1):  
        print( "*", end='')  
    print( )
```

**Output:**

```
>>>
```

```
* * * * *
```

```
* * * *
```

```
* * *
```

```
* *
```

```
*
```

# Python Programming Tutorial

## PYTHON WHILE LOOP

In Python, while loop is used to execute number of statements or body till the specified condition is true. Once the condition is false, the control will come out of the loop.

### Python While Loop Syntax

```
while <expression>:
```

```
    Body
```

Here, loop Body will execute till the expression passed is true. The Body may be a single statement or multiple statement.

### Python While Loop Example 1

```
a=10
while (a>0):
    print( "Value of a is",a)
    a=a-2
print( "Loop is Completed")
```

### Output:

```
>>>
Value of a is 10
Value of a is 8
Value of a is 6
Value of a is 4
Value of a is 2
Loop is Completed
>>>
```

### Explanation:

Firstly, the value in the variable is initialized.

Secondly, the condition/expression in the while is evaluated. Consequently if condition is true, the control enters in the body and executes all the statements . If the condition/expression passed results in false then the control exists the body and straight away control goes to next instruction after body of while.

Thirdly, in case condition was true having completed all the statements, the variable is incremented or decremented. Having changed the value of variable step second is followed. This process continues till the expression/condition becomes false.

Finally Rest of code after body is executed.

### Python While Loop Example 2

```
n=153
sum=0
```

# Python Programming Tutorial

```
while( n>0):  
    r=n%10  
    sum+=r  
    n=n/10  
print( sum)
```

**Output:**

```
>>>  
9  
>>>
```

## PYTHON BREAK

Break statement is a jump statement which is used to transfer execution control. It breaks the current execution and in case of inner loop, inner loop terminates immediately.

When break statement is applied the control points to the line following the body of the loop, hence applying break statement makes the loop to terminate and controls goes to next line pointing after loop body.

### **Python Break Example 1**

```
for i in [1,2,3,4,5]:  
    if i==4:  
        print( "Element found")  
        break  
    print( i,end='')
```

**Output:**

```
>>>  
1 2 3 Element found  
>>>
```

### **Python Break Example 2**

```
for letter in 'Python3':  
    if( letter == 'o'):  
        break  
    print( letter)
```

**Output:**

```
P  
y  
t  
h
```

## PYTHON CONTINUE STATEMENT

# Python Programming Tutorial

Python Continue Statement is a jump statement which is used to skip execution of current iteration. After skipping, loop continue with next iteration.

We can use continue statement with for as well as while loop in Python.

## Python Continue Statement Example

```
a=0
while( a<=5):
    a=a+1
    if a%2==0:
        continue
    print( a)
print( "End of Loop")
```

### Output:

```
>>>
1
3
5
End of Loop
>>>
```

## PYTHON PASS

In Python, pass keyword is used to execute nothing it means, when we don't want to execute code, the pass can be used to execute empty. It is same as the name refers to. It just makes the control to pass by without executing any code. If we want to bypass any code pass statement can be used.

## Python Pass Example

```
for i in [1,2,3,4,5]:
    if i==3:
        pass
    print( "Pass when value is",i )
print( i,end="")
```

### Output:

```
>>>
1 2 Pass when value is 3
3 4 5
```

## PYTHON STRINGS

Python string is a built-in type text sequence. It is used to handle textual data in python. Python Strings are immutable sequences of Unicode points. Creating Strings are simplest and easy to use in Python.

# Python Programming Tutorial

We can simply create Python String by enclosing a text in single as well as double quotes. Python treat both single and double quotes statements same.

## Accessing Python Strings

In Python, Strings are stored as individual characters in a contiguous memory location.

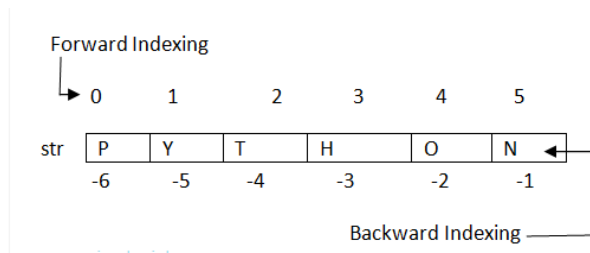
The benefit of using String is that it can be accessed from both the directions (forward and backward).

Both forward as well as backward indexing are provided using Strings in Python.

Forward indexing starts with 0,1,2,3,....

Backward indexing starts with -1,-2,-3,-4,....

Example



## Python String Example

Here, we are creating a simple program to retrieve String in reverse as well as normal form.

```
name="Rajat"
length=len(name)
i=0
for n in range(-1,(-length-1),-1):
    print( name[i],"\t",name[n])
    i+=1
```

**Output:**

```
>>>
```

```
R      t
a      a
j      j
a      a
t      R
```

```
>>>
```

## Python Strings Operators

To perform operation on string, Python provides basically 3 types of Operators that are given below.

- Basic Operators.
- Membership Operators.
- Relational Operators.

# Python Programming Tutorial

## Python String Basic Operators

There are two types of basic operators in String "+" and "\*".

### i. String Concatenation Operator (+)

The concatenation operator (+) concatenates two Strings and creates a new String.

#### Python String Concatenation Example

```
>>> "ratan" + "jaiswal"
```

**Output:**

```
'ratanjaiswal'
```

```
>>>
```

| Expression         | Output        |
|--------------------|---------------|
| '10' + '20'        | '1020'        |
| "s" + "007"        | 's007'        |
| 'abcd123' + 'xyz4' | 'abcd123xyz4' |

**NOTE:** Both the operands passed for concatenation must be of same type, else it will show an error.

**Eg:**

```
'abc' + 3
```

```
>>>
```

**output:**

Traceback (most recent call last):

File "", line 1, in

```
'abc' + 3
```

TypeError: cannot concatenate 'str' and 'int' objects

```
>>>
```

### Python String Replication Operator (\*)

Replication operator uses two parameters for operation, One is the integer value and the other one is the String argument.

The Replication operator is used to repeat a string number of times. The string will be repeated the number of times which is given by the integer value.

#### Python String Replication Example

```
>>> 5*"Vimal"
```

**Output:**

```
'VimalVimalVimalVimalVimal'
```

# Python Programming Tutorial

| Expression | Output       |
|------------|--------------|
| "soono"*2  | 'soonosoono' |
| 3*'1'      | '111'        |
| '\$'*5     | '\$\$\$\$\$' |

**NOTE:** We can use Replication operator in any way i.e., int \* string or string \* int. Both the parameters passed cannot be of same type.

## Python String Membership Operators

Membership Operators are already discussed in the Operators section. Let see with context of String.

There are two types of Membership operators

- 1) in:"in" operator returns true if a character or the entire substring is present in the specified string, otherwise false.
- 2) not in:"not in" operator returns true if a character or entire substring does not exist in the specified string, otherwise false.

## Python String membership operator Example

```
>>> str1="javatpoint"
>>> str2='sssit'
>>> str3="seomount"
>>> str4='java'
>>> st5="it"
>>> str6="seo"
>>> str4 in str1
True
>>> str5 in str2
>>> st5 in str2
True
>>> str6 in str3
True
>>> str4 not in str1
False
>>> str1 not in str4
True
```

## Python Relational Operators

All the comparison (relational) operators i.e., (<,>,<=,>=,==,!=,<>) are also applicable for strings. The Strings are compared based on the ASCII value or Unicode(i.e., dictionary Order).

# Python Programming Tutorial

## Python Relational Operators Example

```
>>> "RAJAT"=="RAJAT"
```

```
True
```

```
>>> "afsha">='Afsha'
```

```
True
```

```
>>> "Z"<>"z"
```

```
True
```

### Explanation:

The ASCII value of a is 97, b is 98, c is 99 and so on. The ASCII value of A is 65, B is 66, C is 67 and so on. The comparison between strings are done on the basis on ASCII value.

## Python String Slice Notation

Python String slice can be defined as a substring which is the part of the string. Therefore further substring can be obtained from a string.

There can be many forms to slice a string, as string can be accessed or indexed from both the direction and hence string can also be sliced from both the directions.

## Python String Slice Syntax

```
<string_name>[startIndex:endIndex],
```

```
<string_name>[:endIndex],
```

```
<string_name>[startIndex:]
```

Python String Slice Example 1

```
>>> str="Nikhil"
```

```
>>> str[0:6]
```

```
'Nikhil'
```

```
>>> str[0:3]
```

```
'Nik'
```

```
>>> str[2:5]
```

```
'khi'
```

```
>>> str[:6]
```

```
'Nikhil'
```

```
>>> str[3:]
```

```
'hil'
```

**Note:** startIndex in String slice is inclusive whereas endIndex is exclusive.

String slice can also be used with Concatenation operator to get whole string.

Python String Slice Example 2

```
>>> str="Mahesh"
```

# Python Programming Tutorial

```
>>> str[:6]+str[6:]  
'Mahesh'  
//here 6 is the length of the string.
```

## Python String Functions and Methods

Python provides various predefined or built-in string functions. They are as follows:

| Methods                                | Descriptions                                                                                                                                           |
|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| capitalize()                           | It capitalizes the first character of the String.                                                                                                      |
| count(string,begin,end)                | It Counts number of times substring occurs in a String between begin and end index.                                                                    |
| endswith(suffix ,begin=0,end=n)        | It returns a Boolean value if the string terminates with given suffix between begin and end.                                                           |
| find(substring ,beginIndex, endIndex)  | It returns the index value of the string where substring is found between begin index and end index.                                                   |
| index(substring, beginIndex, endIndex) | It throws an exception if string is not found and works same as find() method.                                                                         |
| isalnum()                              | It returns True if characters in the string are alphanumeric i.e., alphabets or numbers and there is at least 1 character. Otherwise it returns False. |
| isalpha()                              | It returns True when all the characters are alphabets and there is at least one character, otherwise False.                                            |
| isdigit()                              | It returns True if all the characters are digit and there is at least one character, otherwise False.                                                  |
| islower()                              | It returns True if the characters of a string are in lower case, otherwise False.                                                                      |
| isupper()                              | It returns True if characters of a string are in Upper case, otherwise False.                                                                          |
| isspace()                              | It returns True if the characters of a string are                                                                                                      |

## Python Programming Tutorial

|                                |                                                                                                                   |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|
|                                | whitespace, otherwise false.                                                                                      |
| len(string)                    | It returns the length of a string.                                                                                |
| lower()                        | It converts all the characters of a string to Lower case.                                                         |
| upper()                        | It converts all the characters of a string to Upper Case.                                                         |
| startswith(str ,begin=0,end=n) | It returns a Boolean value if the string starts with given str between begin and end.                             |
| swapcase()                     | It inverts case of all characters in a string.                                                                    |
| lstrip()                       | It removes all leading whitespace of a string and can also be used to remove particular character from leading.   |
| rstrip()                       | It removes all trailing whitespace of a string and can also be used to remove particular character from trailing. |
| split()                        | It split the sentence into list.                                                                                  |
| join()                         | Joins the elements of an iterable to the end of the string                                                        |
| istitle()                      | Returns True if the string follows the rules of a title                                                           |
| Replace()                      | It replaces a specified phrase with another specified phrase.                                                     |

# Python Programming Tutorial

## 1) Python String capitalize() Method Example

This method capitalizes the first character of the String.

```
msg = "welcome to sssit"  
print(msg.capitalize() )
```

**Output:**

```
>>>  
Welcome to sssit  
>>>
```

## 2) Python String count() Method Example

Python String count(string) Method Example

This method counts number of times substring occurs in a String between begin and end index.

```
msg = "welcome to sssit"  
substr1 = "o"  
print( msg.count(substr1, 4, 16) )  
substr2 = "t"  
print( msg.count(substr2) )
```

**Output:**

```
>>>  
2  
2  
>>>
```

## 3) Python String endswith(string) Method Example

This method returns a Boolean value if the string terminates with given suffix between begin and end.

```
string1="Welcome to SSSIT"  
substring1="SSSIT"  
substring2="to"  
substring3="of"  
print( string1.endswith(substring1))  
print( string1.endswith(substring2,2,16))  
print( string1.endswith(substring3,2,19))  
print( string1.endswith(substring3))
```

**Output:**

```
>>>  
True  
False  
False  
False  
>>>
```

# Python Programming Tutorial

## 4) Python String find(string) Method Example

This method returns the index value of the string where substring is found between begin index and end index.

```
str="Welcome to SSSIT"
substr1="come"
substr2="to"
print( str.find(substr1))
print( str.find(substr2))
print( str.find(substr1,3,10))
```

**Output:**

```
>>>
3
8
3
>>>
```

## 5) Python String index() Method Example

This method returns the index value of the string where substring is found between begin index and end index.

```
str="Welcome to world of SSSIT"
substr1="come"
substr2="of"
print( str.index(substr1))
print( str.index(substr2))
print( str.index(substr1,3,10))
print( str.index(substr2,19))
```

**Output:**

```
>>>
3
17
3
Traceback (most recent call last):
  File "C:/Python27/fin.py", line 7, in
    print( str.index(substr2,19)
ValueError: substring not found
>>>
```

# Python Programming Tutorial

## 6) Python String isalnum() Method Example

This method returns True if characters in the string are alphanumeric i.e., alphabets or numbers and there is at least 1 character. Otherwise it returns False.

```
str="Welcome to sssit"
print( str.isalnum())
str1="Python47"
print( str1.isalnum())
```

**Output:**

```
>>>
False
True
>>>
```

## 7) Python String isalpha() Method Example

It returns True when all the characters are alphabets and there is at least one character, otherwise False.

```
string1="HelloPython" # Even space is not allowed
print( string1.isalpha())
string2="This is Python2.7.4"
print( string2.isalpha())
```

**Output:**

```
>>>
True
False
>>>
```

## 8) Python String isdigit() Method Example

This method returns True if all the characters are digit and there is at least one character, otherwise False.

```
string1="HelloPython"
print( string1.isdigit())
string2="98564738"
print( string2.isdigit())
```

**Output:**

```
>>>
False
True
>>>
```

# Python Programming Tutorial

## 9) Python String islower() Method Example

This method returns True if the characters of a string are in lower case, otherwise False.

```
string1="Hello Python"
print( string1.islower())
string2="welcome to "
print( string2.islower())
```

**Output:**

```
>>>
False
True
>>>
```

## 10) Python String isupper() Method Example

This method returns False if characters of a string are in Upper case, otherwise False.

```
string1="Hello Python"
print( string1.isupper())
string2="WELCOME TO"
print( string2.isupper())
```

**Output:**

```
>>>
False
True
>>>
```

## 11) Python String isspace() Method Example

This method returns True if the characters of a string are whitespace, otherwise false.

```
string1="  "
print( string1.isspace())
string2="WELCOME TO WORLD OF PYT"
print( string2.isspace())
```

**Output:**

```
>>>
True
False
>>>
```

## 12) Python String len(string) Method Example

This method returns the length of a string.

# Python Programming Tutorial

```
string1="  "  
print( len(string1))  
string2="WELCOME TO SSSIT"  
print( len(string2))
```

**Output:**

```
>>>  
4  
16  
>>>
```

## 13) Python String lower() Method Example

It converts all the characters of a string to Lower case.

```
string1="Hello Python"  
print( string1.lower())  
string2="WELCOME TO SSSIT"  
print( string2.lower())
```

**Output:**

```
>>>  
hello python  
welcome to sssit  
>>>
```

## 14) Python String upper() Method Example

This method converts all the characters of a string to upper case.

```
string1="Hello Python"  
print( string1.upper())  
string2="welcome to SSSIT"  
print( string2.upper())
```

**Output:**

```
>>>  
HELLO PYTHON  
WELCOME TO SSSIT  
>>>
```

## 15) Python String startswith(string) Method Example

This method returns a Boolean value if the string starts with given str between begin and end.

```
string1="Hello Python"  
print( string1.startswith('Hello'))  
string2="welcome to SSSIT"
```

# Python Programming Tutorial

```
print( string2.startswith('come',3,7))
```

**Output:**

```
>>>
True
True
>>>
```

## 16) Python String swapcase() Method Example

It inverts case of all characters in a string.

```
string1="Hello Python"
print( string1.swapcase())
string2="welcome to SSSIT"
print( string2.swapcase())
```

**Output:**

```
>>>
HELLO pYTHON
WELCOME TO sssit
>>>
```

## 17) Python String lstrip() Method Example

It removes all leading whitespace of a string and can also be used to remove particular character from leading.

```
string1="  Hello Python"
print( string1.lstrip())
string2="@@@@@welcome to SSSIT"
print( string2.lstrip('@'))
```

**Output:**

```
>>>
Hello Python
welcome to world to SSSIT
>>>
```

## 18) Python String rstrip() Method Example

It removes all trailing whitespace of a string and can also be used to remove particular character from trailing.

```
string1="  Hello Python  "
print( string1.rstrip())
string2="@welcome to SSSIT!!!"
print( string2.rstrip('!'))
```

# Python Programming Tutorial

## Output:

```
>>>  
        Hello Python  
@welcome to SSSIT  
>>>
```

### 19) Python String split () Method Example

It split the sentence into different elements of the list.

```
string1=" Welcome to India"  
print( string1.split())
```

## Output:

```
["Welcome", "to", "India"]
```

### 20) Python String join() Method Example

Join all items in a tuple into a string, using a hash character as separator.

```
myTuple = ("John", "Peter", "Vicky")
```

```
x = "#".join(myTuple)
```

```
print(x)
```

## Output:

```
John#Peter#Vicky
```

### 21) Python String istitle() Method Example

Check if each word start with an upper case letter:

```
txt = "Hello, And Welcome To My World!"  
x = txt.istitle()  
print(x)
```

## Output:

```
True
```

### 22) Python String replace() Method Example

```
txt = "I like bananas"  
x = txt.replace("bananas", "apples")  
print(x)
```

## Output:

```
I like apples
```

# Python Programming Tutorial

## PYTHON LIST

Python list is a data structure which is used to store various types of data.

In Python, **lists are mutable** i.e., Python will not create a new list if we modify an element of the list. It works as a container that holds other objects in a given order. We can perform various operations like insertion and deletion on list.

A list can be composed by storing a sequence of different type of values separated by commas.

Python list is enclosed between square([]) brackets and elements are stored in the index basis with starting index 0.

### Python List Example

```
data1=[1,2,3,4]
data2=['x','y','z']
data3=[12.5,11.6]
data4=['raman','rahul']
data5=[]
data6=['abhinav',10,56.4,'a']
```

A list can be created by putting the value inside the square bracket and separated by comma.

### Python List Syntax

<list\_name>=[value1,value2,value3,...,valuen]

Syntax to Access Python List

<list\_name>[index]

Python allows us to access value from the list by various ways.

### Python Accessing List Elements Example

```
data1=[1,2,3,4]
data2=['x','y','z']
print( data1[0])
print( data1[0:2])
print( data2[-3:-1])
print( data1[0:])
print( data2[:2])
```

#### Output:

```
>>>
```

```
>>>
```

```
1
```

```
[1, 2]
```

```
['x', 'y']
```

```
[1, 2, 3, 4]
```

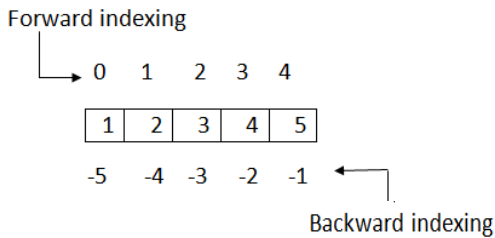
# Python Programming Tutorial

```
['x', 'y']  
>>>
```

## Elements in a Lists:

Following are the pictorial representation of a list. We can see that it allows to access elements from both end (forward and backward).

Data=[1,2,3,4,5]



Data[0]=1=Data[-5] , Data[1]=2=Data[-4] , Data[2]=3=Data[-3] ,  
=4=Data[-2] , Data[4]=5=Data[-1].

## Note: Internal Memory Organization:

List do not store the elements directly at the index. In fact a reference is stored at each index which subsequently refers to the object stored somewhere in the memory. This is due to the fact that some objects may be large enough than other objects and hence they are stored at some other memory location.

## Python List Operations

Apart from creating and accessing elements from the list, Python allows us to perform various other operations on the list. Some common operations are given below

### a) Adding Python Lists

In Python, lists can be added by using the concatenation operator(+) to join two lists.

#### Add lists Example 1

```
list1=[10,20]  
list2=[30,40]  
list3=list1+list2  
print( list3)
```

#### Output:

```
>>>  
[10, 20, 30, 40]  
>>>
```

Note: '+' operator implies that both the operands passed must be list else error will be shown.

# Python Programming Tutorial

## Add lists Example 2

```
list1=[10,20]
list1+30
print( list1 )
```

### Output:

```
Traceback (most recent call last):
  File "C:/Python27/lis.py", line 2, in <module>
    list1+30
```

## b) Python Replicating lists

Replicating means repeating, It can be performed by using '\*' operator by a specific number of time.

### Python list Replication Example

```
list1=[10,20]
print( list1*2)
```

### Output:

```
>>>
[10, 20, 10, 20]
>>>
```

## c) Python List Slicing

A subpart of a list can be retrieved on the basis of index. This subpart is known as list slice. This feature allows us to get sub-list of specified start and end index.

### Python List Slicing Example

```
list1=[1,2,4,5,7]
print( list1[0:2])
print( list1[4])
list1[1]=9
print( list1)
```

### Output:

```
>>>
[1, 2]
7
[1, 9, 4, 5, 7]
>>>
```

**Note:** If the index provided in the list slice is outside the list, then it raises an Index Error exception.

## Python List Other Operations

Apart from above operations various other functions can also be performed on List such as Updating, Appending and Deleting elements from a List.

# Python Programming Tutorial

## Python Updating List

To update or change the value of particular index of a list, assign the value to that particular index of the List.

### Python Updating List Example

```
data1=[5,10,15,20,25]
print( "Values of list are: ")
print( data1)
data1[2]="Multiple of 5"
print( "Values of list are: ")
print( data1)
Output:
>>>
Values of list are:
[5, 10, 15, 20, 25]
Values of list are:
[5, 10, 'Multiple of 5', 20, 25]
>>>
```

## Appending Python List

Python provides, append() method which is used to append i.e., add an element at the end of the existing elements.

### Python Append List Example

```
list1=[10,"rahul",'z']
print( "Elements of List are: ")
print( list1)
list1.append(10.45)
print( "List after appending: ")
print( list1)
Output:
>>>
Elements of List are:
[10, 'rahul', 'z']
List after appending:
[10, 'rahul', 'z', 10.45]
>>>
```

## Deleting Elements

# Python Programming Tutorial

In Python, del statement can be used to delete an element from the list. It can also be used to delete all items from startIndex to endIndex.

## Python delete List Example

```
list1=[10,'rahul',50.8,'a',20,30]
print( list1)
del list1[0]
print( list1)
del list1[0:3]
print( list1)
```

### Output:

```
>>>
[10, 'rahul', 50.8, 'a', 20, 30]
['rahul', 50.8, 'a', 20, 30]
[20, 30]
>>>
```

## Python lists Method

Python provides various Built-in functions and methods for Lists that we can apply on the list. Following are the common list functions.

| Function  | Description                                       |
|-----------|---------------------------------------------------|
| min(list) | It returns the minimum value from the list given. |
| max(list) | It returns the largest value from the given list. |
| len(list) | It returns number of elements in a list.          |

### 1) Python List min() method Example

This method is used to get min value from the list (only numbers).

```
list1=[101,981,189,228,105.22]
list2=[701,981,989,928,105.22]
print( "Minimum value in List1: ",min(list1))
print( "Minimum value in List2: ",min(list2))
```

### Output:

```
>>>
Minimum value in List1: 101
Minimum value in List2: 105.22
>>>
```

### 2) Python List max() method Example

# Python Programming Tutorial

This method is used to get max value from the list.

```
list1=[101,981,189,228,105.22]
list2=[701,981,989,928,105.22]
print( "Maximum value in List : ",max(list1))
print( "Maximum value in List : ",max(list2))
```

**Output:**

```
>>>
Maximum value in List : 981
Maximum value in List : 989
>>>
```

### 3) Python List len() method Example

This method is used to get length of the the list.

```
list1=[101,981,'abcd','xyz','m']
list2=['aman','shekhar',100.45,98.2]
print( "No. of elements in List1: ",len(list1) )
print( "No. of elements in List2: ",len(list2))
```

**Output:**

```
>>>
No. of elements in List1 : 5
No. of elements in List2 : 4
>>>
```

### 4) Python List list(sequence) method Example

This method is used to form a list from the given sequence of elements.

```
seq=(145,"abcd",'a')
data=list(seq)
print( "List formed is : ",data)
```

**Output:**

```
>>>
List formed is : [145, 'abcd', 'a']
>>>
```

# Python Programming Tutorial

There are following built-in methods of List

| Methods              | Description                                                                               |
|----------------------|-------------------------------------------------------------------------------------------|
| index(object)        | It returns the index value of the object.                                                 |
| count(object)        | It returns the number of times an object is repeated in list.                             |
| pop()/pop(index)     | It returns the last object or the specified indexed object. It removes the popped object. |
| insert(index,object) | It inserts an object at the given index.                                                  |
| extend(sequence)     | It adds the sequence to existing list.                                                    |
| remove(object)       | It removes the object from the given List.                                                |
| reverse()            | It reverses the position of all the elements of a list.                                   |
| sort()               | It is used to sort the elements of the List.                                              |

## 1) Python List index() Method Example

```
data = [786,'abc','a',123.5]
print( "Index of 123.5:", data.index(123.5))
print( "Index of a is", data.index('a'))
```

**Output:**

```
>>>
Index of 123.5 : 3
Index of a is 2
>>>
```

## 2) Python List count(object) Method Example

```
data = [786,'abc','a',123.5,786,'rahul','b',786]
print( "Number of times 123.5 occurred is", data.count(123.5))
print( "Number of times 786 occurred is", data.count(786))
```

**Output:**

```
>>>
Number of times 123.5 occurred is 1
Number of times 786 occurred is 3
>>>
```

## 3) Python List pop()/pop(int) Method Example

```
data = [786,'abc','a',123.5,786]
print( "Last element is", data.pop())
```

# Python Programming Tutorial

```
print( "2nd position element:", data.pop(1))  
print( data )
```

**Output:**

```
>>>  
Last element is 786  
2nd position element:abc  
[786, 'a', 123.5]  
>>>
```

## 4) Python List insert(index,object) Method Example

```
data=['abc',123,10.5,'a']  
data.insert(2,'hello')  
print( data)
```

**Output:**

```
>>>  
['abc', 123, 'hello', 10.5, 'a']  
>>>
```

## 5) Python List extend(sequence) Method Example

```
data1=['abc',123,10.5,'a']  
data2=['ram',541]  
data1.extend(data2)  
print( data1)  
print( data2 )
```

**Output:**

```
>>>  
['abc', 123, 10.5, 'a', 'ram', 541]  
['ram', 541]  
>>>
```

## 6) Python List remove(object) Method Example

```
data1=['abc',123,10.5,'a','xyz']  
data2=['ram',541]  
print( data1)  
data1.remove('xyz')  
print( data1)  
print( data2 )  
data2.remove('ram')  
print( data2)
```

# Python Programming Tutorial

## Output:

```
>>>
['abc', 123, 10.5, 'a', 'xyz']
['abc', 123, 10.5, 'a']
['ram', 541]
[541]
>>>
```

## 7) Python List reverse() Method Example

```
list1=[10,20,30,40,50]
list1.reverse()
print( list1 )
```

## Output:

```
>>>
[50, 40, 30, 20, 10]
>>>
```

## 8) Python List sort() Method Example

```
list1=[10,50,13,'rahul','aakash']
list1.sort()
print( list1 )
```

## Output:

```
>>>
[10, 13, 50, 'aakash', 'rahul']
>>>
```

## PYTHON TUPLE

A tuple is a sequence of immutable objects, therefore tuple cannot be changed. It can be used to collect different types of object.

The objects are enclosed within parenthesis and separated by comma.

Tuple is similar to list. Only the difference is that list is enclosed between square bracket, tuple between parenthesis and List has mutable objects whereas Tuple has immutable objects.

## Python Tuple Example

```
>>> data=(10,20,'ram',56.8)
>>> data2="a",10,20.9
>>> data
(10, 20, 'ram', 56.8)
```

# Python Programming Tutorial

```
>>> data2
('a', 10, 20.9)
>>>
```

**NOTE:** If Parenthesis is not given with a sequence, it is by default treated as Tuple.

There can be an empty Tuple also which contains no object. Let's see an example of empty tuple.

## Python Empty Tuple Example

```
tuple1=()
```

Python Single Object Tuple Example

For a single valued tuple, there must be a comma at the end of the value.

```
Tuple1=(10,)
```

## Python Tuple of Tuples Example

Tuples can also be nested, it means we can pass tuple as an element to create a new tuple. See, the following example in which we have created a tuple that contains tuples and the object.

```
tpl1='a','mahesh',10.56
tpl2=tuple1,(10,20,30)
print( tpl1)
print( tpl2)
```

**Output:**

```
>>>
('a', 'mahesh', 10.56)
(('a', 'mahesh', 10.56), (10, 20, 30))
>>>
```

## Accessing Tuple

Accessing of tuple is pretty easy we can access tuple in the same way as List. See, the following example.

## Accessing Tuple Example

```
data1=(1,2,3,4)
data2=('x','y','z')
print( data1[0])
print( data1[0:2])
print( data2[-3:-1])
print( data1[0:] )
print( data2[:2])
```

**Output:**

# Python Programming Tutorial

```
>>>
1
(1, 2)
('x', 'y')
(1, 2, 3, 4)
('x', 'y')
>>>
```

## Elements in a Tuple

```
Data= (1,2,3,4,5,10,19,17)
```

## Elements In A Tuple

```
Data[0]=1=Data[-8] , Data[1]=2=Data[-7] , Data[2]=3=Data[-6] ,
    Data[3]=4=Data[-5] , Data[4]=5=Data[-4] , Data[5]=10=Data[-3],
Data[6]=19=Data[-2],Data[7]=17=Data[-1]
```

## Python Tuple Operations

Python allows us to perform various operations on the tuple. Following are the common tuple operations.

### Adding Tuples Example

Tuple can be added by using the concatenation operator(+) to join two tuples.

```
data1=(1,2,3,4)
data2=('x','y','z')
data3=data1+data2
print( data1)
print( data2 )
print( data3 )
```

#### Output:

```
>>>
(1, 2, 3, 4)
('x', 'y', 'z')
(1, 2, 3, 4, 'x', 'y', 'z')
>>>
```

**Note:** The new sequence formed is a new Tuple.

### Replicating Tuple Example

Replicating means repeating. It can be performed by using '\*' operator by a specific number of time.

```
tuple1=(10,20,30)
tuple2=(40,50,60)
```

# Python Programming Tutorial

```
print( tuple1*2)
print( tuple2*3)
```

**Output:**

```
>>>
(10, 20, 30, 10, 20, 30)
(40, 50, 60, 40, 50, 60, 40, 50, 60)
>>>
```

## Python Tuple Slicing Example

A subpart of a tuple can be retrieved on the basis of index. This subpart is known as tuple slice.

```
data1=(1,2,4,5,7)
print( data1[0:2])
print( data1[4])
print( data1[: -1])
print( data1[-5:])
print( data1)
```

**Output:**

```
>>>
(1, 2)
7
(1, 2, 4, 5)
(1, 2, 4, 5, 7)
(1, 2, 4, 5, 7)
>>>
```

**Note:** If the index provided in the Tuple slice is outside the list, then it raises an IndexError exception.

## Python Tuple other Operations

### Updating elements in a List

Elements of the Tuple cannot be updated. This is due to the fact that Tuples are immutable. Whereas the Tuple can be used to form a new Tuple.

### Example

```
data=(10,20,30)
data[0]=100
print( data)
```

**Output:**

```
>>>
Traceback (most recent call last):
  File "C:/Python27/t.py", line 2, in
```

# Python Programming Tutorial

```
data[0]=100
TypeError: 'tuple' object does not support item assignment
>>>
```

## Creating Tuple from Existing Example

We can create a new tuple by assigning the existing tuple, see the following example.

```
data1=(10,20,30)
data2=(40,50,60)
data3=data1+data2
print( data3)
```

### Output:

```
>>>
(10, 20, 30, 40, 50, 60)
>>>
```

## Python Tuple Deleting Example

Deleting individual element from a tuple is not supported. However the whole of the tuple can be deleted using the del statement.

```
data=(10,20,'rahul',40.6,'z')
print( data)
del data    #will delete the tuple data
print( data) #will show an error since tuple data is already deleted
```

### Output:

```
>>>
(10, 20, 'rahul', 40.6, 'z')
Traceback (most recent call last):
  File "C:/Python27/t.py", line 4, in
    print( data
NameError: name 'data' is not defined
>>>
```

## Functions of Tuple

There are following in-built Type Functions

| Function        | Description                                  |
|-----------------|----------------------------------------------|
| min(tuple)      | It returns the minimum value from a tuple.   |
| max(tuple)      | It returns the maximum value from the tuple. |
| len(tuple)      | It gives the length of a tuple               |
| tuple(sequence) | It converts the sequence into tuple.         |

# Python Programming Tutorial

## Python Tuple min(tuple) Method Example

This method is used to get min value from the sequence of tuple.

```
data=(10,20,55,40.6,87)
print( min(data))
```

**Output:**

```
>>>
10
>>>
```

## Python Tuple max(tuple) Method Example

This method is used to get max value from the sequence of tuple.

```
data=(10,20,98,40.6,55)
print( max(data) )
```

**Output:**

```
>>>
98
>>>
```

## Python Tuple len(tuple) Method Example

This method is used to get length of the tuple.

```
data=(10,20,'rahul',40.6,'z')
print( len(data) )
```

**Output:**

```
>>>
5
>>>
```

**tuple(sequence):**

Eg:

```
dat=[10,20,30,40]
data=tuple(dat)
print( data )
```

**Output:**

```
>>>
(10, 20, 30, 40)
>>>
```

# Python Programming Tutorial

## Why should we use Tuple? (Advantages of Tuple)

Processing of Tuples are faster than Lists.

It makes the data safe as Tuples are immutable and hence cannot be changed.

Tuples are used for String formatting.

## PYTHON DICTIONARY

Dictionary is an unordered set of key and value pair. It is a container that contains data, enclosed within curly braces.

The pair i.e., key and value is known as item. The key passed in the item must be unique.

The key and the value is separated by a colon(:). This pair is known as item. Items are separated from each other by a comma(,). Different items are enclosed within a curly brace and this forms Dictionary.

## Python Dictionary Example

```
data={100:'Ravi' ,101:'Vijay' ,102:'Rahul'}  
print( data)
```

### Output:

```
>>>  
{100: 'Ravi', 101: 'Vijay', 102: 'Rahul'}  
>>>
```

**Note: Dictionary is mutable i.e., value can be updated.**

Key must be unique and immutable. Value is accessed by key. Value can be updated while key cannot be changed.

Dictionary is known as Associative array since the Key works as Index and they are decided by the user.

## Python Dictionary Example

```
plant={}  
plant[1]='Ravi'  
plant[2]='Manoj'  
plant['name']='Hari'  
plant[4]='Om'  
print( plant[2] )  
print( plant['name'])  
print( plant[1])  
print( plant)
```

# Python Programming Tutorial

## Output:

```
>>>
Manoj
Hari
Ravi
{1: 'Ravi', 2: 'Manoj', 4: 'Om', 'name': 'Hari'}
>>>
```

## Accessing Dictionary Values

Since Index is not defined, a Dictionary values can be accessed by their keys only. It means, to access dictionary elements we need to pass key, associated to the value.

## Python Accessing Dictionary Element Syntax

```
<dictionary_name>[key]
</dictionary_name>
Accessing Elements Example
```

```
data1={'Id':100, 'Name':'Suresh', 'Profession':'Developer'}
data2={'Id':101, 'Name':'Ramesh', 'Profession':'Trainer'}
print( "Id of 1st employer is",data1['Id'])
print( "Id of 2nd employer is",data2['Id'])
print( "Name of 1st employer:",data1['Name'])
print( "Profession of 2nd employer:",data2['Profession'])
```

## Output:

```
>>>
Id of 1st employer is 100
Id of 2nd employer is 101
Name of 1st employer is Suresh
Profession of 2nd employer is Trainer
>>>
```

## Updating Python Dictionary Elements

The item i.e., key-value pair can be updated. Updating means new item can be added. The values can be modified.

## Example

```
data1={'Id':100, 'Name':'Suresh', 'Profession':'Developer'}
data2={'Id':101, 'Name':'Ramesh', 'Profession':'Trainer'}
data1['Profession']='Manager'
data2['Salary']=20000
data1['Salary']=15000
```

# Python Programming Tutorial

```
print( data1 )  
print( data2 )
```

## Output:

```
>>>  
{'Salary': 15000, 'Profession': 'Manager', 'Id': 100, 'Name': 'Suresh'}  
{'Salary': 20000, 'Profession': 'Trainer', 'Id': 101, 'Name': 'Ramesh'}  
>>>
```

## Deleting Python Dictionary Elements Example

del statement is used for performing deletion operation.

An item can be deleted from a dictionary using the key only.

## Delete Syntax

```
del <dictionary_name>[key]  
</dictionary_name>
```

Whole of the dictionary can also be deleted using the del statement.

## Example

```
data={100:'Ram', 101:'Suraj', 102:'Alok'}  
del data[102]  
print( data)  
del data  
print( data) #will show an error since dictionary is deleted.
```

## Output:

```
>>>  
{100: 'Ram', 101: 'Suraj'}
```

Traceback (most recent call last):

```
File "C:/Python27/dict.py", line 5, in  
    print( data
```

NameError: name 'data' is not defined

```
>>>
```

## Python Dictionary Functions and Methods

Python Dictionary supports the following Functions

# Python Programming Tutorial

| Functions                    | Description                                         |
|------------------------------|-----------------------------------------------------|
| <code>len(dictionary)</code> | It returns number of items in a dictionary.         |
| <code>str(dictionary)</code> | It gives the string representation of a dictionary. |

## Python Dictionary Methods

| Methods                                                        | Description                                                                                                                                                                                             |
|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>keys()</code>                                            | It returns all the keys element of a dictionary.                                                                                                                                                        |
| <code>values()</code>                                          | It returns all the values element of a dictionary.                                                                                                                                                      |
| <code>items()</code>                                           | It returns all the items(key-value pair) of a dictionary.                                                                                                                                               |
| <code>update(dictionary2)</code>                               | It is used to add items of dictionary2 to first dictionary.                                                                                                                                             |
| <code>clear()</code>                                           | It is used to remove all items of a dictionary. It returns an empty dictionary.                                                                                                                         |
| <code>fromkeys(sequence,value1)/<br/>fromkeys(sequence)</code> | It is used to create a new dictionary from the sequence where sequence elements forms the key and all keys share the values ?value1?. In case value1 is not give, it set the values of keys to be none. |
| <code>copy()</code>                                            | It returns an ordered copy of the data.                                                                                                                                                                 |
| <code>get(key)</code>                                          | It returns the value of the given key. If key is not present it returns none.                                                                                                                           |

### Python Dictionary `len(dictionary)` Example

It returns length of the dictionary.

```
data={100:'Ram', 101:'Suraj', 102:'Alok'}  
print( data)  
print( len(data) )
```

#### Output:

```
>>>  
{100: 'Ram', 101: 'Suraj', 102: 'Alok'}  
3  
>>>
```

### Python Dictionary `str(dictionary)` Example

# Python Programming Tutorial

This method returns string formation of the value.

```
data1={100:'Ram', 101:'Suraj', 102:'Alok'}  
print( str(data1))
```

**Output:**

```
>>>  
{100: 'Ram', 101: 'Suraj', 102: 'Alok'}  
>>>
```

## Python Dictionary keys() Method Example

This method returns all the keys element of a dictionary.

```
data1={100:'Ram', 101:'Suraj', 102:'Alok'}  
print( data1.keys())
```

**Output:**

```
>>>  
[100, 101, 102]  
>>>
```

## Python Dictionary values() Method Example

This method returns all the values element of a dictionary.

```
data1={100:'Ram', 101:'Suraj', 102:'Alok'}  
print( data1.values())
```

Output:

```
>>>  
['Ram', 'Suraj', 'Alok']  
>>>
```

## Python Dictionary items() Method Example

This method returns all the items(key-value pair) of a dictionary.

```
data1={100:'Ram', 101:'Suraj', 102:'Alok'}  
print( data1.items())
```

**Output:**

```
>>>  
[(100, 'Ram'), (101, 'Suraj'), (102, 'Alok')]  
>>>
```

## Python Dictionary update(dictionary2) Method Example

This method is used to add items of dictionary2 to first dictionary.

# Python Programming Tutorial

```
data1={100:'Ram', 101:'Suraj', 102:'Alok'}
data2={103:'Sanjay'}
data1.update(data2)
print( data1)
print( data2)
```

## Output:

```
>>>
{100: 'Ram', 101: 'Suraj', 102: 'Alok', 103: 'Sanjay'}
{103: 'Sanjay'}
>>>
```

## Python Dictionary clear() Method Example

It returns an ordered copy of the data.

```
data1={100:'Ram', 101:'Suraj', 102:'Alok'}
print( data1)
data1.clear()
print( data1)
```

## Output:

```
>>>
{100: 'Ram', 101: 'Suraj', 102: 'Alok'}
{}
>>>
```

## Python Dictionary fromkeys(sequence)/ fromkeys(seq,value) Method Example

This method is used to create a new dictionary from the sequence where sequence elements forms the key and all keys share the values ?value1?. In case value1 is not give, it set the values of keys to be none.

```
sequence=('Id' , 'Number' , 'Email')
data={}
data1={}
data=data.fromkeys(sequence)
print( data)
data1=data1.fromkeys(sequence,100)
print( data1)
```

## Output:

```
>>>
{'Email': None, 'Id': None, 'Number': None}
{'Email': 100, 'Id': 100, 'Number': 100}
>>>
```

# Python Programming Tutorial

## Python Dictionary copy() Method Example

This method returns an ordered copy of the data.

```
data={'Id':100 , 'Name':'Aakash' , 'Age':23}  
data1=data.copy()  
print( data1)
```

### Output:

```
>>>  
{'Age': 23, 'Id': 100, 'Name': 'Aakash'}  
>>>
```

## Python Dictionary get(key) Method Example

This method returns the value of the given key. If key is not present it returns none.

```
data={'Id':100 , 'Name':'Aakash' , 'Age':23}  
print( data.get('Age'))  
print( data.get('Email'))
```

### Output:

```
>>>  
23  
None  
>>>
```

## PYTHON FUNCTIONS

A Function is a self-block of code which is used to organize the functional code.

Function can be called as a section of a program that is written once and can be executed whenever required in the program, thus making code reusability.

Function is a subprogram that works on data and produces some output.

### Types of Functions:

There are two types of Functions.

- a) Built-in Functions: Functions that are predefined and organized into a library. We have used many predefined functions in Python.
- b) User- Defined: Functions that are created by the programmer to meet the requirements.

### Defining a Function

A Function defined in Python should follow the following format:

# Python Programming Tutorial

- 1) Keyword def is used to start and declare a function. Def specifies the starting of function block.
- 2) def is followed by function-name followed by parenthesis.
- 3) Parameters are passed inside the parenthesis. At the end a colon is marked.

## Python Function Syntax

```
def <function_name>(parameters):  
</function_name>
```

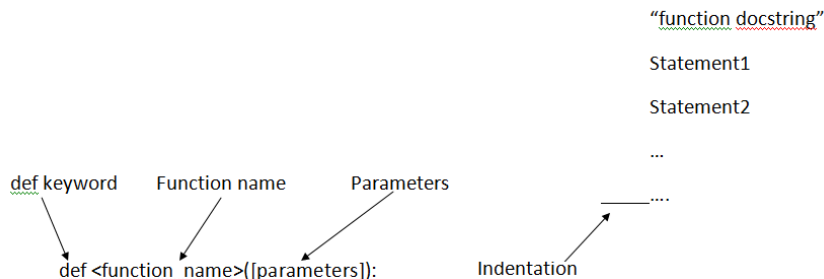
### Example

```
def sum(a,b):
```

- 4) Python code requires indentation (space) of code to keep it associate to the declared block.
- 5) The first statement of the function is optional. It is ?Documentation string? of function.
- 6) Following is the statement to be executed.

### Syntax:

Syntax:



## Invoking a Python Function

To execute a function it needs to be called. This is called function calling.

Function Definition provides the information about function name, parameters and the definition what operation is to be performed. In order to execute the function definition, we need to call the function.

## Python Function Syntax

```
<function_name>(parameters)  
</function_name>
```

### Python Function Example

# Python Programming Tutorial

```
sum(a,b)
```

Here, sum is the function and a, b are the parameters passed to the function definition.

Let's have a look over an example.

## Python Function Example 2

#Providing Function Definition

```
def sum(x,y):  
    "Going to add x and y"  
    s=x+y  
    print( "Sum of two numbers is")  
    print( s)  
    #Calling the sum Function  
    sum(10,20)  
    sum(20,30)
```

**Output:**

```
>>>  
Sum of two numbers is  
30  
Sum of two numbers is  
50  
>>>
```

**NOTE:** Function call will be executed in the order in which it is called.

## Python Function return Statement

return[expression] is used to return response to the caller function. We can use expression with the return keyword. send back the control to the caller with the expression.

In case no expression is given after return it will return None.

In other words return statement is used to exit the function definition.

## Python Function return Example

```
def sum(a,b):  
    "Adding the two values"  
    print( "Print(ing within Function)")  
    print( a+b )  
    return a+b  
def msg():  
    print( "Hello")
```

# Python Programming Tutorial

```
total=sum(10,20)
print( "Print(ing Outside: ",total
msg()
print( "Rest of code"
```

## Output:

```
>>>
Print(ing within Function
30
Print(ing outside: 30
Hello
Rest of code
>>>
```

## Python Function Argument and Parameter

There can be two types of data passed in the function.

- 1) The First type of data is the data passed in the function call. This data is called “arguments”.
- 2) The second type of data is the data received in the function definition. This data is called ‘parameters’.

Arguments can be literals, variables and expressions. Parameters must be variable to hold incoming values.

Alternatively, arguments can be called as actual parameters or actual arguments and parameters can be called as formal parameters or formal arguments.

## Python Function Example

```
def addition(x,y):
    print( x+y)
x=15
addition(x ,10)
addition(x,x)
y=20
addition(x,y)
```

## Output:

```
>>>
25
30
```

# Python Programming Tutorial

35

>>>

## Passing Parameters

Apart from matching the parameters, there are other ways of matching the parameters.

Python supports following types of formal argument:

- 1) Positional argument (Required argument).
- 2) Default argument.
- 3) Keyword argument (Named argument)

## Positional/Required Arguments:

When the function call statement must match the number and order of arguments as defined in the function definition. It is Positional Argument matching.

Python Function Positional Argument Example

#Function definition of sum

```
def sum(a,b):  
    "Function having two parameters"  
    c=a+b  
    print( c )
```

```
sum(10,20)
```

```
sum (20)
```

## Output:

>>>

30

Traceback (most recent call last):

File "C:/Python27/su.py", line 8, in <module>

sum(20)

TypeError: sum() takes exactly 2 arguments (1 given)

>>>

</module>

## Explanation:

# Python Programming Tutorial

1) In the first case, when `sum()` function is called passing two values i.e., 10 and 20 it matches with function definition parameter and hence 10 and 20 is assigned to `a` and `b` respectively. The sum is calculated and `print(ed)`.

2) In the second case, when `sum()` function is called passing a single value i.e., 20 , it is passed to function definition. Function definition accepts two parameters whereas only one value is being passed, hence it will show an error.

## Python Function Default Arguments

Default Argument is the argument which provides the default values to the parameters passed in the function definition, in case value is not provided in the function call default value is used.

## Python Function Default Argument Example

```
#Function Definition
def msg(Id,Name,Age=21):
    print("Printing the passed value" )
    print( Id)
    print( Name)
    print( Age)
    return

#Function call
msg(Id=100,Name='Ravi',Age=20)
msg(Id=101,Name='Ratan')
```

### Output:

```
>>>
100
Ravi
20
101
Ratan
21
>>>
```

### Explanation:

1) In first case, when `msg()` function is called passing three different values i.e., 100 , Ravi and 20, these values will be assigned to respective parameters and thus respective values will be `print(ed)`.

2) In second case, when `msg()` function is called passing two values i.e., 101 and Ratan, these values will be assigned to `Id` and `Name` respectively. No value is assigned for third argument via function call and hence it will retain its default value i.e, 21.

# Python Programming Tutorial

## Python Keyword Arguments

Using the Keyword Argument, the argument passed in function call is matched with function definition on the basis of the name of the parameter.

## Python keyword Argument Example

```
def msg(id,name):  
    print("Printing passed value")  
    print( id)  
    print( name)  
    return  
msg(id=100,name='Raj')  
msg(name='Rahul',id=101)
```

### Output:

```
>>>  
100  
Raj  
101  
Rahul  
>>>
```

### Explanation:

1) In the first case, when msg() function is called passing two values i.e., id and name the position of parameter passed is same as that of function definition and hence values are initialized to respective parameters in function definition. This is done on the basis of the name of the parameter.

2) In second case, when msg() function is called passing two values i.e., name and id, although the position of two parameters is different it initialize the value of id in Function call to id in Function Definition. same with name parameter. Hence, values are initialized on the basis of name of the parameter.

## Python Anonymous Function

Anonymous Functions are the functions that are not bond to name. It means anonymous function does not has a name.

Anonymous Functions are created by using a keyword "lambda".

Lambda takes any number of arguments and returns an evaluated expression.

Lambda is created without using the def keyword.

# Python Programming Tutorial

## Python Anonymous Function Syntax

```
lambda arg1,args2,args3,?,argsn :expression
```

Python Anonymous Function Example

```
#Function Definiton
```

```
square=lambda x1: x1*x1
```

```
#Calling square as a function
```

```
print( "Square of number is",square(10)
```

**Output:**

```
>>>
```

```
Square of number is 100
```

```
>>>
```

Difference between Normal Functions and Anonymous Function:

Have a look over two examples:

**Example:**

**Normal function:**

```
#Function Definiton
```

```
def square(x):
```

```
    return x*x
```

```
#Calling square function
```

```
print( "Square of number is",square(10))
```

Anonymous function:

```
#Function Definiton
```

```
square=lambda x1: x1*x1
```

```
#Calling square as a function
```

```
print( "Square of number is",square(10))
```

Explanation:

Anonymous is created without using def keyword.

lambda keyword is used to create anonymous function.

It returns the evaluated expression.

# Python Programming Tutorial

## Scope of Variable:

Scope of a variable can be determined by the part in which variable is defined. Each variable cannot be accessed in each part of a program. There are two types of variables based on Scope:

1) Local Variable.

2) Global Variable.

## 1) Python Local Variables

Variables declared inside a function body is known as Local Variable. These have a local access thus these variables cannot be accessed outside the function body in which they are declared.

## Python Local Variables Example

```
def msg():  
    a=10  
    print( "Value of a is",a)  
    return
```

```
msg()  
print( a #it will show error since variable is local
```

## Output:

```
>>>  
Value of a is 10
```

Traceback (most recent call last):

```
File "C:/Python27/lam.py", line 7, in <module>  
    print( a #it will show error since variable is local  
NameError: name 'a' is not defined  
>>>  
</module>
```

## b) Python Global Variable

Variable defined outside the function is called Global Variable. Global variable is accessed all over program thus global variable have widest accessibility.

## Python Global Variable Example

```
b=20
```

# Python Programming Tutorial

```
def msg():  
    a=10  
    print( "Value of a is",a)  
    print( "Value of b is",b)  
    return  
  
    msg()  
    print( b)
```

## Output:

```
>>>  
Value of a is 10  
Value of b is 20  
20  
>>>
```

## PYTHON MODULE

Modules are used to categorize Python code into smaller parts. A module is simply a Python file, where classes, functions and variables are defined. Grouping similar code into a single file makes it easy to access. Have a look at below example.

If the content of a book is not indexed or categorized into individual chapters, the book might have turned boring and hectic. Hence, dividing book into chapters made it easy to understand.

In the same sense python modules are the files which have similar code. Thus module is simplifying a python code where classes, variables and functions are defined.

## Python Module Advantage

Python provides the following advantages for using module:

- 1) Reusability: Module can be used in some other python code. Hence it provides the facility of code reusability.
- 2) Categorization: Similar type of attributes can be placed in one module.

## Importing a Module:

There are different ways by which you we can import a module. These are as follows:

### 1) Using import statement:

"import" statement can be used to import a module.

### Syntax:

# Python Programming Tutorial

```
import <file_name1, file_name2,...file_name(n)="">
</file_name1,>
```

## Example

```
def add(a,b):
    c=a+b
    print( c)
    return
```

Save the file by the name addition.py. To import this file "import" statement is used.

```
import addition
addition.add(10,20)
addition.add(30,40)
```

Create another python file in which you want to import the former python file. For that, import statement is used as given in the above example. The corresponding method can be used by file\_name.method (). (Here, addition.add (), where addition is the python file and add () is the method defined in the file addition.py)

## Output:

```
>>>
30
70
>>>
```

**NOTE:** You can access any function which is inside a module by module name and function name separated by dot. It is also known as period. Whole notation is known as dot notation.

## Python Importing Multiple Modules Example

1) msg.py:

```
def msg_method():
    print( "Today the weather is rainy")
    return
```

2) display.py:

```
def display_method():
    print( "The weather is Sunny")
    return
```

3) multiimport.py:

```
import msg,display
msg.msg_method()
display.display_method()
```

# Python Programming Tutorial

## Output:

```
>>>
Today the weather is rainy
The weather is Sunny
>>>
```

## 2) Using from.. import statement:

from..import statement is used to import particular attribute from a module. In case you do not want whole of the module to be imported then you can use from ?import statement.

### Syntax:

```
from <module_name> import <attribute1,attribute2,attribute3,...attributen>
</attribute1,attribute2,attribute3,...attributen></module_name>
Python from.. import Example
```

```
def circle(r):
    print( 3.14*r*r)
    return
```

```
def square(l):
    print( l*l)
    return
```

```
def rectangle(l,b):
    print( l*b)
    return
```

```
def triangle(b,h):
    print( 0.5*b*h)
    return
```

2) area1.py

```
from area import square,rectangle
square(10)
rectangle(2,5)
```

Output:

```
>>>
100
10
>>>
```

# Python Programming Tutorial

## 3) To import whole module:

You can import whole of the module using "from? import \*"

Syntax:

```
from <module_name> import *  
</module_name>
```

Using the above statement all the attributes defined in the module will be imported and hence you can access each attribute.

1) area.py

Same as above example

2) area1.py

```
from area import *  
square(10)  
rectangle(2,5)  
circle(5)  
triangle(10,20)
```

Output:

```
>>>  
100  
10  
78.5  
100.0  
>>>
```

## Built in Modules in Python:

There are many built in modules in Python. Some of them are as follows:

math, random , threading , collections , os , mailbox , string , time , tkinter etc..

Each module has a number of built in functions which can be used to perform various functions.

Let's have a look over each module:

### 1) math:

Using math module , you can use different built in mathematical functions.

### Functions:

# Python Programming Tutorial

| Function         | Description                                                   |
|------------------|---------------------------------------------------------------|
| ceil(n)          | It returns the next integer number of the given number        |
| sqrt(n)          | It returns the Square root of the given number.               |
| exp(n)           | It returns the natural logarithm e raised to the given number |
| floor(n)         | It returns the previous integer number of the given number.   |
| log(n,baseto)    | It returns the natural logarithm of the number.               |
| pow(baseto, exp) | It returns baseto raised to the exp power.                    |
| sin(n)           | It returns sine of the given radian.                          |
| cos(n)           | It returns cosine of the given radian.                        |
| tan(n)           | It returns tangent of the given radian.                       |

## Python Math Module Example

```
import math
a=4.6
print( math.ceil(a))
print( math.floor(a))
b=9
print( math.sqrt(b))
print( math.exp(3.0))
print( math.log(2.0))
print( math.pow(2.0,3.0))
print( math.sin(0))
print( math.cos(0))
print( math.tan(45))
```

### Output:

```
>>>
5.0
4.0
3.0
20.0855369232
0.69314718056
8.0
0.0
1.0
1.61977519054
>>>
```

## 2) random:

# Python Programming Tutorial

The random module is used to generate the random numbers. It provides the following two built in functions:

| Function     | Description                                                                      |
|--------------|----------------------------------------------------------------------------------|
| random()     | It returns a random number between 0.0 and 1.0 where 1.0 is exclusive.           |
| randint(x,y) | It returns a random number between x and y where both the numbers are inclusive. |

## Python Module Example

```
import random
```

```
print( random.random())  
print( random.randint(2,8))
```

### Output:

```
>>>  
0.797473843839  
7  
>>>
```

## PYTHON EXCEPTION HANDLING

Exception can be said to be any abnormal condition in a program resulting to the disruption in the flow of the program.

Whenever an exception occurs the program halts the execution and thus further code is not executed. Thus exception is that error which python script is unable to tackle with.

Exception in a code can also be handled. In case it is not handled, then the code is not executed further and hence execution stops when exception occurs.

## Common Exceptions

**ZeroDivisionError:** Occurs when a number is divided by zero.  
**NameError:** It occurs when a name is not found. It may be local or global.  
**IndentationError:** If incorrect indentation is given.  
**IOError:** It occurs when Input Output operation fails.  
**EOFError:** It occurs when end of the file is reached and yet operations are being performed.

## Exception Handling:

# Python Programming Tutorial

The suspicious code can be handled by using the try block. Enclose the code which raises an exception inside the try block. The try block is followed by except statement. It is then further followed by statements which are executed during exception and in case if exception does not occur.

## Syntax:

```
try:
    malicious code
except Exception1:
    execute code
except Exception2:
    execute code
....
....
except ExceptionN:
    execute code
else:
    In case of no exception, execute the else block code.
```

Python Exception Handling Example

```
try:
    a=10/0
    print( a )
except ArithmeticError:
    print( "This statement is raising an exception")
else:
    print( "Welcome")
```

## Output:

```
>>>
This statement is raising an exception
>>>
```

## Explanation:

The malicious code (code having exception) is enclosed in the try block.

Try block is followed by except statement. There can be multiple except statement with a single try block.

Except statement specifies the exception which occurred. In case that exception is occurred, the corresponding statement will be executed.

At the last you can provide else statement. It is executed when no exception is occurred.

Python Exception(Except with no Exception) Example

Except statement can also be used without specifying Exception.

# Python Programming Tutorial

## Syntax:

```
try:
    code
except:
    code to be executed in case exception occurs.
else:
    code to be executed in case exception does not occur.
```

## Example

```
try:
    a=10/0
except:
    print("Arithmetic Exception")
else:
    print("Successfully Done")
```

## Output:

```
>>>
Arithmetic Exception
>>>
```

## Declaring Multiple Exception in Python

Python allows us to declare multiple exceptions using the same except statement.

## Syntax:

```
try:
    code
except Exception1,Exception2,Exception3,...,ExceptionN
    execute this code in case any Exception of these occur.
else:
    execute code in case no exception occurred.
```

## Example

```
try:
    a=10/0
except ArithmeticError,StandardError:
    print("Arithmetic Exception")
else:
    print("Successfully Done")
```

# Python Programming Tutorial

## Output:

```
>>>
Arithmetic Exception
>>>
```

## Finally Block:

In case if there is any code which the user want to be executed, whether exception occurs or not then that code can be placed inside the finally block. Finally block will always be executed irrespective of the exception.

## Syntax:

```
try:
    Code
finally:
    code which is must to be executed.
```

## Example

```
try:
    a=10/0
    print( "Exception occurred")
finally:
    print( "Code to be executed")
```

## Output:

```
>>>
Code to be executed
Traceback (most recent call last):
  File "C:/Python27/noexception.py", line 2, in <module>
    a=10/0
ZeroDivisionError: integer division or modulo by zero
>>>
```

In the above example finally block is executed. Since exception is not handled therefore exception occurred and execution is stopped.

## Raise an Exception:

You can explicitly throw an exception in Python using *raise* statement. *raise* will cause an exception to occur and thus execution control will stop in case it is not handled.

## Syntax:

# Python Programming Tutorial

raise Exception\_class,<value>

## Example

```
try:
    a=10
    print( a)
    raise NameError("Hello")
except NameError as e:
    print( "An exception occurred")
    print( e)
```

## Output:

```
>>>
10
An exception occurred
Hello
>>>
```

## Explanation:

- i) To raise an exception, raise statement is used. It is followed by exception class name.
- ii) Exception can be provided with a value that can be given in the parenthesis. (here, Hello)
- iii) To access the value "as" keyword is used. "e" is used as a reference variable which stores the value of the exception.

## PYTHON FILEHANDLING

File handling is an important part of any web application.

Python has several functions for creating, reading, updating, and deleting files.

### File Handling

- The key function for working with files in Python is the open() function.
- The open () function takes two parameters filename, and mode.

There are four different methods (modes) for opening a file:

1. "r" - Read - Default value. Opens a file for reading, error if the file does not exist

# Python Programming Tutorial

2. "a" - Append - Opens a file for appending, creates the file if it does not exist
3. "w" - Write - Opens a file for writing, creates the file if it does not exist
4. "x" - Create - Creates the specified file, returns an error if the file exists

In addition you can specify if the file should be handled as binary or text mode

- "t" - Text - Default value. Text mode
- "b" - Binary - Binary mode (e.g. images)

## Syntax

To open a file for reading it is enough to specify the name of the file:

```
f = open("demofile.txt")
```

The code above is the same as:

```
f = open("demofile.txt", "rt")
```

Because "r" for read, and "t" for text are the default values, you do not need to specify them.

**Note: Make sure the file exists, or else you will get an error.**

Assume we have the following file, located in the same folder as Python:

### demofile.txt

*Hello! Welcome to demofile.txt*

*This file is for testing purposes.*

*Good Luck!*

To open the file, use the built-in open() function.

The open() function returns a file object, which has a read() method for reading the content of the file:

## Example

```
f = open("demofile.txt", "r")
```

```
print(f.read())
```

Read Only Parts of the File

By default the read() method returns the whole text, but you can also specify how many character you want to return:

## Example

# Python Programming Tutorial

Return the 5 first characters of the file:

```
f = open("demofile.txt", "r")
print(f.read(5))
```

## Read Lines

You can return one line by using the `readline()` method:

### Example

Read one line of the file:

```
f = open("demofile.txt", "r")
print(f.readline())
```

By calling `readline()` two times, you can read the two first lines:

### Example

Read two lines of the file:

```
f = open("demofile.txt", "r")
print(f.readline())
print(f.readline())
```

By looping through the lines of the file, you can read the whole file, line by line:

### Example

Loop through the file line by line:

```
f = open("demofile.txt", "r")
for x in f:
    print(x)
```

## Write to an Existing File

To write to an existing file, you must add a parameter to the `open()` function:

- "a" - Append - will append to the end of the file
- "w" - Write - will overwrite any existing content

### Example

Open the file "demofile.txt" and append content to the file:

```
f = open("demofile.txt", "a")
f.write("Now the file has one more line!")
```

# Python Programming Tutorial

## Example

Open the file "demofile.txt" and overwrite the content:

```
f = open("demofile.txt", "w")
```

```
f.write("Woops! I have deleted the content!")
```

Note: the "w" method will overwrite the entire file.

## Create a New File

To create a new file in Python, use the open() method, with one of the following parameters:

- "x" - Create - will create a file, returns an error if the file exist
- "a" - Append - will create a file if the specified file does not exist
- "w" - Write - will create a file if the specified file does not exist

## Example

Create a file called "myfile.txt":

```
f = open("myfile.txt", "x")
```

Result: a new empty file is created!

## Example

Create a new file if it does not exist:

```
f = open("myfile.txt", "w")
```

## Delete a File

To delete a file, you must import the OS module, and run its os.remove() function:

## Example

Remove the file "demofile.txt":

```
import os
```

```
os.remove("demofile.txt")
```

Check if File exist:

To avoid getting an error, you might want to check if the file exist before you try to delete it:

## Example

Check if file exist, then delete it:

# Python Programming Tutorial

```
import os
if os.path.exists("demofile.txt"):
    os.remove("demofile.txt")
else:
    print("The file does not exist")
```

## Delete Folder

To delete an entire folder, use the `os.rmdir()` method:

### Example

Remove the folder "myfolder":

```
import os
os.rmdir("myfolder")
```

Note: You can only remove empty folders.